CGE-P LAB CURRICULUM · V2

Lab 5.2: AWS Security Services Baseline

CloudTrail records what happened. Config records what the resource looked like. Security Hub aggregates findings into one normalized view. Athena is what turns any of it into an answer.

That last sentence is the whole reason this lab matters. **This is where AU-6 is earned.** Lab 2.3 ships logs into a bucket, which is AU-3 and AU-11. It stops short of AU-6, audit review and analysis, because nothing there reads a log. Here, something does.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/05_02_aws_security_services.md` in your clone, not from the PDF.

Learning objectives

- Deploy a multi-region CloudTrail with log-file validation and KMS encryption.
- Enable S3 **data events** on the evidence vault, and understand why that bucket specifically.
- Enable Security Hub with the NIST 800-53 Rev 5 standard.
- **Query the trail with Athena and produce an answer**, which is what AU-6 costs.
- Defend a control mapping you disagree with.

Controls implemented

Control	Service	Note
AU-2, AU-12	CloudTrail management events	Audit event selection and generation.
AU-3	CloudTrail data events on the vault	Object-level, JSON, guaranteed delivery.
AU-6	**Athena queries over the trail**	The control most often claimed on the strength of collection alone.
AU-9	Log-file validation digests + trail bucket versioning	
AU-11	Lifecycle on the trail bucket	
SC-28	Trail encrypted with a CMK	
RA-5, SI-4	Security Hub	
CM-2, CM-6, CM-8	AWS Config, where deployable	

A mapping to argue with

Log-file validation is commonly mapped to **AU-10 (non-repudiation)**. Push back on that.

Log-file validation emits an hourly digest file signed by an AWS-managed key, letting you detect modification or deletion of log files. That is integrity protection of audit records: **AU-9**, and specifically AU-9(3) when the mechanism is cryptographic, with a strong SI-7 argument alongside it.

AU-10 is about binding an action to an individual so they cannot later deny performing it. CloudTrail's `userIdentity` block does contribute to that. The *digest file* does not; it proves the log was not altered, not that a particular human did a particular thing.

Both mappings can be argued. AU-9 is the stronger argument, and the reason to teach this is not the answer. It is that **a control mapping is a claim you defend, and an assessor who disagrees with yours will ask you why**. Being able to say "we mapped it to AU-9 because the digest protects the record rather than binding the actor, and here is where AU-10 is covered instead" is the skill.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.5** vault deployed. Data events target it.
- AWS account with admin or near-admin rights.
- Terraform `>= 1.10`, using the Lab 2.2 backend.
- Check for an existing Security Hub subscription you do not want to disturb: `aws securityhub describe-hub`.

Estimated time & cost

THIS LAB COSTS REAL MONEY. Read this before applying.

- CloudTrail **management** events: the first trail is free.
- CloudTrail **data** events: about **$0.10 per 100,000 events**. Scoped to the evidence vault only, a lab account generates a handful. Point them at a busy bucket and this is the line item that surprises people.
- Security Hub: about $0.001 per check per month. The NIST 800-53 standard runs roughly 300 checks, so under $1/month in a quiet single-account lab.
- AWS Config: about $2/month per recorder plus $0.001 per rule evaluation. Many Security Hub controls need Config to evaluate.
- Trail CMK: **$1/month**.
- S3 storage for logs: pennies, and bounded by the lifecycle rule.

Destroy within an hour and expect under $2. Leave Security Hub, Config, and two CMKs running for a month and it is **$10 to $15**. Estimates that quote $5 to $8 are usually leaving out Config and the second key.

Architecture

```
every region
------------
CloudTrail "cgep-lab-mgmt"  multi-region
  management events                      AU-2 / AU-12
  data events: s3://EVIDENCE_VAULT/*     AU-3
```

```
    log-file validation -> digests           AU-9 / SI-7
    KMS CMK                                  SC-28
        |
        v
  S3 trail bucket: versioned, KMS, TLS-only, lifecycle 400d
        |
        +--> Athena external table  ----> AU-6 queries
        |
  AWS Config recorder  ---> same bucket      CM-2 / CM-6 / CM-8   (SCP may block)
        |
        v
  Security Hub (NIST 800-53 Rev 5 + FSBP)    RA-5 / SI-4
```

Step-by-step walkthrough

Concept: why baseline services beat point tools

Every cloud security vendor will sell you a slicker console. CloudTrail, Config, and Security Hub are inside the platform you already pay for, mapped to the controls assessors ask about, and emitting JSON you can pipe into the pipeline you already built. They are not pretty. They are durable. Pick the durable one.

Step 1 The trail bucket, on the shared baseline

It is easy to hold this bucket to a lower standard than a workload bucket, on the reasoning that nothing but CloudTrail writes to it. That is exactly backwards: it carries the audit record of the entire account.

```
resource "aws_kms_key" "trail" {
  description             = "CMK for CloudTrail logs"
  enable_key_rotation     = true
  deletion_window_in_days = 7
  policy                  = data.aws_iam_policy_document.trail_kms.json
}

data "aws_iam_policy_document" "trail_kms" {
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }

  # CloudTrail must be able to encrypt log files with this key.
  statement {
    sid       = "AllowCloudTrailEncrypt"
```

```
    effect    = "Allow"
    actions   = ["kms:GenerateDataKey*", "kms:DescribeKey"]
    resources = ["*"]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringLike"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }

  # Athena and your analysts must be able to read them back.
  statement {
    sid       = "AllowAccountDecrypt"
    effect    = "Allow"
    actions   = ["kms:Decrypt", "kms:DescribeKey"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "trail" {
  bucket = "cgep-lab-cloudtrail-${random_id.suffix.hex}"
}

resource "aws_s3_bucket_ownership_controls" "trail" {
  bucket = aws_s3_bucket.trail.id
  rule { object_ownership = "BucketOwnerEnforced" }
}

resource "aws_s3_bucket_versioning" "trail" {
  bucket = aws_s3_bucket.trail.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "trail" {
  bucket = aws_s3_bucket.trail.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.trail.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "trail" {
  bucket                  = aws_s3_bucket.trail.id
  block_public_acls       = true
```

```
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# AU-11: 400 days covers a full annual audit cycle plus the audit itself.
resource "aws_s3_bucket_lifecycle_configuration" "trail" {
  bucket     = aws_s3_bucket.trail.id
  depends_on = [aws_s3_bucket_versioning.trail]

  rule {
    id     = "expire-trail-logs"
    status = "Enabled"
    filter {}

    transition {
      days          = 90
      storage_class = "GLACIER_IR"
    }

    expiration { days = var.trail_retention_days }

    noncurrent_version_expiration { noncurrent_days = 30 }
  }
}
```

The Glacier Instant Retrieval transition at 90 days is new. Trail logs are written constantly, read rarely, and must be readable immediately when they are read. That is exactly the storage class's use case, and it cuts the dominant cost of a long retention.

Step 2 The trail bucket policy

```
data "aws_iam_policy_document" "trail" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.trail.arn, "${aws_s3_bucket.trail.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  statement {
    sid       = "AWSCloudTrailAclCheck"
    effect    = "Allow"
```

```
    actions   = ["s3:GetBucketAcl"]
    resources = [aws_s3_bucket.trail.arn]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }

  statement {
    sid       = "AWSCloudTrailWrite"
    effect    = "Allow"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.trail.arn}/AWSLogs/${local.account_id}/*"]
    principals {
      type        = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "s3:x-amz-acl"
      values   = ["bucket-owner-full-control"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceArn"
      values   = ["arn:${local.partition}:cloudtrail:${var.aws_region}:${local.account_id}:trail/cgep-
lab-mgmt"]
    }
  }
}

resource "aws_s3_bucket_policy" "trail" {
  bucket = aws_s3_bucket.trail.id
  policy = data.aws_iam_policy_document.trail.json
}
```

The `aws:SourceArn` conditions are confused-deputy protection: without them, someone else's trail in another account could be pointed at your bucket. Note that this is the same pattern Lab 2.3 uses for S3 access-log delivery, which is why v2 uses a policy there rather than an ACL.

Step 3 The trail, with data events

```
resource "aws_cloudtrail" "mgmt" {
  name                          = "cgep-lab-mgmt"
  s3_bucket_name                = aws_s3_bucket.trail.id
  is_multi_region_trail         = true
```

```
  include_global_service_events = true
  enable_log_file_validation    = true
  kms_key_id                    = aws_kms_key.trail.arn

  # AU-3: object-level events on the evidence vault ONLY.
  # This is the one bucket where "who read the evidence" is itself an audit
  # question. Scoping it here keeps the bill in cents instead of dollars.
  advanced_event_selector {
    name = "S3 data events on the evidence vault"

    field_selector {
      field  = "eventCategory"
      equals = ["Data"]
    }
    field_selector {
      field  = "resources.type"
      equals = ["AWS::S3::Object"]
    }
    field_selector {
      field       = "resources.ARN"
      starts_with = ["${var.evidence_vault_arn}/"]
    }
  }

  advanced_event_selector {
    name = "All management events"

    field_selector {
      field  = "eventCategory"
      equals = ["Management"]
    }
  }

  depends_on = [aws_s3_bucket_policy.trail]
}
```

Why the vault and nothing else. Skipping data events entirely leaves object-level access to the evidence vault unrecorded, and that is the gap an insider uses: read the evidence, learn what the auditor will see, and no record exists that you looked. Management events show the bucket being *created*, never that it was *read*.

Scoping data events to one bucket is the honest middle. Turning them on account-wide in a real environment is a real budget decision, and saying so is better than pretending the choice is free.

Step 4 Security Hub

```
resource "aws_securityhub_account" "this" {}

resource "aws_securityhub_standards_subscription" "nist_800_53" {
  standards_arn = "arn:${local.partition}:securityhub:${var.aws_region}::standards/nist-800-53/v/
5.0.0"
  depends_on    = [aws_securityhub_account.this]
}
```

```
resource "aws_securityhub_standards_subscription" "fsbp" {
  standards_arn = "arn:${local.partition}:securityhub:${var.aws_region}::standards/aws-foundational-
security-best-practices/v/1.0.0"
  depends_on    = [aws_securityhub_account.this]
}
```

If Security Hub is already enabled:

```
terraform import aws_securityhub_account.this "$(aws sts get-caller-identity --query Account --output
text)"
```

A prediction to check. Point Security Hub at an account holding buckets built the easy way, with ACLs enabled, no TLS-deny policy and versioning off, and expect findings S3.12, S3.5 and S3.14 respectively. Point it at the buckets from Lab 2.3 and those three are absent.

Watching that happen costs one screenshot and is the most direct evidence you will get that the Chapter 2 baseline and the Chapter 5 monitoring agree with each other.

Step 5 AWS Config, and the SCP wall

Config is included, and in many org-managed accounts it is centrally administered and blocked by a service control policy:

```
AccessDeniedException: ... is not authorized to perform: config:PutConfigurationRecorder
... with an explicit deny in a service control policy
```

If that is you, comment out the Config resources. Then note what Security Hub does next: it raises a CRITICAL finding titled "AWS Config should be enabled and use the service-linked role for resource recording."

That finding is itself the evidence. Your account is reporting its own gap, in a machine-readable format, without you writing anything. Capture it, cite it in your write-up as a known and accepted limitation with the compensating control named (Config is managed at the org level), and you have handled it the way a GRC engineer should. Silence would have been the wrong answer; so would pretending you deployed it.

Step 6 Apply and wait

```
export AWS_PROFILE=cgep
terraform init && terraform apply -auto-approve
```

Wait 10 to 20 minutes. Security Hub populates its first findings slowly.

Step 7 AU-6, for real: Athena over the trail

This is the step that separates "we collect logs" from "we review them."

```
resource "aws_athena_workgroup" "grc" {
  name = "cgep-grc"

  configuration {
    enforce_workgroup_configuration = true

    result_configuration {
      output_location = "s3://${aws_s3_bucket.trail.id}/athena-results/"

      encryption_configuration {
        encryption_option = "SSE_KMS"
        kms_key_arn       = aws_kms_key.trail.arn
      }
    }
  }
}

resource "aws_glue_catalog_database" "trail" {
  name = "cgep_grc"
}
```

Create the table. CloudTrail's Athena DDL is long and AWS generates it for you from the console; the partition-projection version below is worth using because it avoids running `MSCK REPAIR TABLE` forever:

```
CREATE EXTERNAL TABLE cgep_grc.cloudtrail_logs (
  eventVersion STRING,
  userIdentity STRUCT<
    type: STRING, principalId: STRING, arn: STRING, accountId: STRING,
    userName: STRING,
    sessionContext: STRUCT<
      attributes: STRUCT<mfaAuthenticated: STRING, creationDate: STRING>>>,
  eventTime STRING,
  eventSource STRING,
  eventName STRING,
  awsRegion STRING,
  sourceIPAddress STRING,
  userAgent STRING,
  errorCode STRING,
  errorMessage STRING,
  requestParameters STRING,
  resources ARRAY<STRUCT<ARN: STRING, accountId: STRING, type: STRING>>,
  readOnly STRING,
  eventType STRING
)
PARTITIONED BY (region STRING, dt STRING)
ROW FORMAT SERDE 'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT 'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://<TRAIL_BUCKET>/AWSLogs/<ACCOUNT_ID>/CloudTrail/'
TBLPROPERTIES (
  'projection.enabled' = 'true',
  'projection.region.type' = 'enum',
  'projection.region.values' = 'us-east-1,us-west-2',
  'projection.dt.type' = 'date',
```

```
  'projection.dt.range' = '2026/01/01,NOW',
  'projection.dt.format' = 'yyyy/MM/dd',
  'projection.dt.interval' = '1',
  'projection.dt.interval.unit' = 'DAYS',
  'storage.location.template' =
    's3://<TRAIL_BUCKET>/AWSLogs/<ACCOUNT_ID>/CloudTrail/${region}/${dt}/'
);
```

Now the three queries that constitute an audit review. Save them; they are the deliverable.

```
-- AU-6.1: who touched the evidence vault, and what did they do?
SELECT eventTime, userIdentity.arn, eventName, sourceIPAddress, errorCode
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '7' day, '%Y/%m/%d')
  AND eventSource = 's3.amazonaws.com'
  AND element_at(resources, 1).ARN LIKE '%evidence-vault%'
ORDER BY eventTime DESC;

-- AU-6.2: console sign-ins without MFA. AC-2 / IA-2 evidence.
SELECT eventTime, userIdentity.arn, sourceIPAddress
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '30' day, '%Y/%m/%d')
  AND eventName = 'ConsoleLogin'
  AND userIdentity.sessionContext.attributes.mfaAuthenticated = 'false'
ORDER BY eventTime DESC;

-- AU-6.3: denied actions, clustered by principal. The signal that someone
-- is probing the edges of their permissions.
SELECT userIdentity.arn, eventName, count(*) AS attempts
FROM cgep_grc.cloudtrail_logs
WHERE dt >= date_format(current_date - interval '7' day, '%Y/%m/%d')
  AND errorCode IN ('AccessDenied', 'UnauthorizedOperation')
GROUP BY userIdentity.arn, eventName
HAVING count(*) > 5
ORDER BY attempts DESC;
```

```
mkdir -p evidence/lab-5-2
aws athena start-query-execution \
  --work-group cgep-grc \
  --query-string "$(cat queries/au6-vault-access.sql)" \
  --query-execution-context Database=cgep_grc
```

AU-6 is a process, not a resource. Running the query once satisfies nothing. What satisfies AU-6 is: the query exists, it runs on a defined cadence, a named human reviews the output, and the review is recorded. Schedule it (EventBridge on a weekly rule is enough for the capstone), write the output into your evidence vault, and name the reviewer in your write-up.

If you do not schedule it, say so honestly and mark AU-6 as `partial` in your OSCAL. A partial you can defend beats an `implemented` you cannot.

Step 8 Capture findings as evidence

```
aws securityhub get-findings --region us-east-1 --max-results 50 \
  > evidence/lab-5-2/security-hub-findings.json

aws cloudtrail get-trail-status --name cgep-lab-mgmt --region us-east-1 \
  > evidence/lab-5-2/trail-status.json

# AU-9: prove the digest chain validates
aws cloudtrail validate-logs \
  --trail-arn "$(terraform output -raw trail_arn)" \
  --start-time "$(date -u -d '1 day ago' +%Y-%m-%dT%H:%M:%SZ)" \
  | tee evidence/lab-5-2/log-validation.txt
```

That last command is the AU-9 evidence, and it is the one most often skipped. It walks the digest chain and reports whether any log file was modified or deleted. Enabling validation and never validating is the same shape of mistake as writing a policy and never testing it.

Verification

- `aws cloudtrail get-trail-status` returns `"IsLogging": true`.
- `aws cloudtrail get-event-selectors` shows both advanced selectors.
- `aws cloudtrail validate-logs` reports no invalid files.
- `aws securityhub describe-hub` returns the hub ARN.
- At least one finding within 30 minutes.
- All three Athena queries return results, or an explainable empty set.
- Read an object from the evidence vault, wait, then confirm the read appears in the AU-6.1 query output. **That round trip is the proof that data events work.**

Portfolio submission checklist

- [ ] `terraform/baselines/aws/` with `cloudtrail.tf`, `security_hub.tf`, `athena.tf`, optionally `config.tf`.
- [ ] `queries/*.sql`, all three AU-6 queries.
- [ ] README mapping services to controls, **including the AU-9-versus-AU-10 argument**.
- [ ] `evidence/lab-5-2/security-hub-findings.json`
- [ ] `evidence/lab-5-2/log-validation.txt`
- [ ] `evidence/lab-5-2/au6-review-output.json`, plus a note on cadence and reviewer.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`InsufficientS3BucketPolicyException`.** The bucket policy is missing an `aws:SourceArn` condition, or the trail is being created before the policy. The `depends_on` sequences it.
- **`InsufficientEncryptionPolicyException`.** The trail CMK policy lacks the `cloudtrail.amazonaws.com GenerateDataKey*` grant.
- **`ResourceConflictException: Account is already subscribed`.** Import: `terraform import aws_securityhub_account.this <ACCOUNT_ID>`.
- **`AccessDeniedException ... explicit deny in a service control policy`** for Config. Centrally managed. Comment it out and cite the Security Hub finding as evidence of the gap.
- **Athena returns zero rows.** Check the `LOCATION` includes the account ID and that `projection.dt.range` starts before your trail did. Partition projection silently returns nothing for a range with no data.
- **Athena `HIVE_CURSOR_ERROR` on decryption.** The querying principal needs `kms:Decrypt` on the trail CMK. That is the `AllowAccountDecrypt` statement.
- **No findings after 30 minutes.** Check `aws securityhub get-enabled-standards`.
- **Config recorder name conflict.** One recorder per region. Delete the existing one first.

Cleanup

```
# Capture evidence BEFORE destroying.
aws securityhub get-findings --region us-east-1 --max-results 50 \
  > evidence/lab-5-2/security-hub-findings.json

# Keep Security Hub enabled but drop it from state, if you want it running.
terraform state rm aws_securityhub_account.this

terraform destroy -auto-approve
```

The trail bucket holds objects and is versioned, so add `force_destroy = true` or empty it with the version-aware loop from Lab 2.3. Both CMKs enter their deletion windows and bill throughout.

If ongoing cost worries you, the highest-impact action is unsubscribing the Security Hub standards. The hub is free; the checks bill.

> **The starter's audit gaps.** `GAPS.md` closes with two the account-level controls here bear on directly: **GAP-08**, an API Gateway stage with no access logging, no throttling and no WAF, and **GAP-06**, a Lambda with no DLQ and no X-Ray. CloudTrail gives you who called the API; neither replaces the per-service logging those gaps are actually missing, and saying so precisely is the difference between a real control narrative and a hopeful one.

How this feeds the capstone

Capstone Layer 1 requires a multi-region CloudTrail with log-file validation writing to a dedicated bucket. That is Step 3, and the trail bucket is Step 1.

Your OSCAL component declares `implemented-requirement` entries for AU-2, AU-3, AU-6, AU-9, RA-5, and SI-4. The implementation statements name CloudTrail, Athena, and Security Hub. The evidence URIs point at signed copies of `security-hub-findings.json`, `log-validation.txt`, and your AU-6 query output in the vault.

Be careful with AU-6 specifically. If you scheduled the review, claim `implemented` and cite the EventBridge rule. If you ran it once by hand, claim `partial` and say what is missing. The grader is reading for exactly that distinction, because claiming AU-6 on the strength of collection alone is the single most common overclaim in this space.

Revision history

v2 (current)

- AU-6 is now implemented rather than claimed: a Glue database, an Athena workgroup and three review queries, with a note that running them once by hand is `partial`, not `implemented`.
- CloudTrail data events enabled for the evidence vault, scoped by an advanced event selector. Object-level reads of the vault were previously unrecorded.
- The trail bucket moved to a customer-managed key with versioning, lifecycle including a Glacier Instant Retrieval transition, and a TLS-deny policy.
- Log-file validation remapped from AU-10 to AU-9(3) and SI-7, with the argument for the change stated rather than assumed.
- Added `aws cloudtrail validate-logs` to the evidence steps, so validation is exercised rather than merely enabled.
- Cost estimate corrected upward to $10 to $15 per month running, from $5 to $8.

v1

Initial release: management events only, AES256 trail bucket, AU-6 claimed without a review mechanism, log-file validation mapped to AU-10 and never run.